

Multi-agent env

Real designs rarely have one bus, your testbench env often owns several agents, one per interface or protocol role

Env, N agents, and shared checkers

- The environment is the parent that builds agents and shared components like scoreboard or
- Each agent still follows the one-protocol rule
- Active versus passive is still per agent via is active in ConfigDB
- In connect phase
- Stimulus may run in parallel on each agent

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Multi-agent env

INTERACTIVE TOOL

Multi-agent env

Sketch an environment with **two+ agents** and a **shared scoreboard**: each monitor's analysis port fans in. Starter: UART + SPI agents, both analysis links up.

Starter example: env holds UART and SPI agents plus a shared scoreboard; both monitor analysis ports are connected.

Load starter example

Challenges 0 / 22 cleared

Quiz: env: A multi-agent environment typically...

☐ owns two or more agents plus shared checkers like a scoreboard

☐ replaces the DUT with a Makefile

☐ only runs in report_phase

☐ disables all TLM ports

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

env

Parent that owns agents and shared checkers.

N agents

One agent per interface / protocol role.

shared sb

One scoreboard; many analysis fans-in.

connect

Wire each mon.ap → sb export in connect_phase.

Lab

SCENARIO

FOCUS AGENT

Multi-agent env lab starter

Real UVM literacy

```
❯ wsl - module16-uvm-multi-agent/examples/multi-agent-sketch

# Track A - real Unix
# real shell session (Track A - multi-agent env)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module16-uvm-multi-agent

$ ls examples/multi-agent-sketch/
env.txt

$ cat examples/multi-agent-sketch/env.txt
Multi-agent UVM env sketch (Literacy)
-----

Hierarchy:
└─ uvm_test
   └─ uvm_env
      ├── uart_agent (active/passive per is_active)
      │   ├── sequencer / driver / monitor
      │   └── vif + UART interface
      ├── spi_agent
      │   ├── sequencer / driver / monitor
      │   └── vif + SPI interface
      └── scoreboard (shared)
          ├── actual_imp from uart monitor.analysis_port
          └── actual_imp from spi monitor.analysis_port
              (or separate expected/actual per stream)

build_phase (env):
uart_agent = uart_agent::type_id::create("uart_agent", this);
spi_agent = spi_agent::type_id::create("spi_agent", this);
scoreboard = my_scoreboard::type_id::create("scoreboard", this);

connect_phase (env):
uart_agent.monitor.analysis_port.connect(scoreboard.uart_imp);
spi_agent.monitor.analysis_port.connect(scoreboard.spi_imp);
// driver/sequencer TLM inside each agent as usual

ConfigDB (typical - per agent path):
uvm_config_db<virtual uart_if>::set(this, "uart_agent", "vif", uart_vif);
uvm_config_db<virtual spi_if>::set(this, "spi_agent", "vif", spi_vif);
uvm_config_db<uvm_active_passive_enum>::set(this, "uart_agent", "is_active", UVM_ACTIVE);
uvm_config_db<uvm_active_passive_enum>::set(this, "spi_agent", "is_active", UVM_PASSIVE);

Why multi-agent:
DUT has multiple interfaces - one agent each
shared scoreboard sees traffic from all monitors
virtual sequences (next module) coordinate parallel stimulus

Common mistakes:
two protocols crammed into one agent
forgot to connect one monitor's analysis_port
one global is_active for unlike agents
scoreboard expects one stream but two unrelated protocols mixed

Legacy reference:
learn_uvm2017_sv_verilator/module6/examples/multi_agent/multi_agent_sv

$ grep -n "MA_Env\|MA_Agent\|uvm_env\|analysis_port"
./../learn_uvm2017_sv_verilator/module6/examples/multi_agent/multi_agent_sv | head -12
14: * └─ MA_Env (uvm_env)
15: * └─ MA_Agent a0 (agent_id=0)
16: * └─ MA_Agent a1 (agent_id=1)
90: * MA_Agent: Agent for multi-agent example
107: class MA_Agent extends uvm_agent;
112: 'uvm_component_utils(MA_Agent)
291: * MA_Env: Environment for multi-agent example
301: class MA_Env extends uvm_env;
302: MA_Agent a0; // Agent 0
303: MA_Agent a1; // Agent 1
305: 'uvm_component_utils(MA_Env)
341: a0 = MA_Agent::type_id::create("a0", this);
```

Real shell — multi-agent sketch

Pitfalls to watch

- Do not put two unrelated protocols in one agent, add a second agent instead
- Do not forget to connect every monitor you care about
- Do not assume one agent's ConfigDB settings apply to another
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, disconnect one analysis link and explain what breaks
- On real UVM, sketch env with two agents and one shared scoreboard
- When you are ready